

Rapport Final de Tests

Projet de plateforme de recommandation de films

Groupe: [Numéro du groupe]

Projet: Plateforme de recommandation de films

Étudiants: [Noms]

Contents

1	Introduction	2
2	Méthodologie de test	2
2.1	Environnement	2
2.2	Outils utilisés	2
2.3	Approche	2
3	Résumé des tests	2
4	Tests unitaires	3
4.1	Détail des rapports d'exécution (PHPUnit)	4
5	Tests de robustesse	5
6	Tests de performance	7
7	Tests de sécurité	7
8	Analyse des défauts détectés	8
9	Indicateurs de métriques	9
10	Discussion et interprétation	9
10.1	Le système est-il stable ?	9
10.2	Le système est-il sécurisé ?	9
10.3	Quels sont les risques ?	10
11	Recommandations	10
11.1	Amélioration des performances	10
11.2	Correction des problèmes de sécurité	10
11.3	Renforcement de la validation des données	11
12	Conclusion	11

1 Introduction

Ce document présente l'exécution et les résultats des tests réalisés sur la plateforme de recommandation de films.

Contrairement au PVT, ce document contient les résultats réels des tests exécutés.

Contexte du projet :

La plateforme permet aux utilisateurs de rechercher des films, consulter leurs détails et recevoir des recommandations personnalisées basées sur leurs préférences et leur historique.

Objectifs du rapport :

- Présenter les tests exécutés
- Analyser les résultats obtenus
- Identifier les anomalies
- Évaluer la qualité du système

2 Méthodologie de test

2.1 Environnement

Les tests ont été exécutés en environnement local sous XAMPP avec base de données MySQL.

2.2 Outils utilisés

- PHPUnit – tests unitaires PHP
- Python (aiohttp) – tests de performance
- XAMPP – serveur local
- MySQL – base de données
- Navigateur web – tests fonctionnels et AJAX

2.3 Approche

- Tests automatisés (PHPUnit, Python)
- Tests manuels pour l'interface utilisateur
- Tests de charge et de robustesse
- Tests de sécurité (vérification des injections SQL)

3 Résumé des tests

Total	Réussis	Échoués
70+	50+	20+

Observation : La majorité des tests sont réussis avec quelques anomalies détectées dans la robustesse et le scoring.

4 Tests unitaires

Résultats :

- **Base de données :** Gestion correcte des films, des favoris et des avis (opérations CRUD validées).
- **Scoring :** Fonctionnement global correct, mais incohérences observées dans certains cas complexes liés aux acteurs.
- **Interface :** Les principaux éléments d'interaction (recherche dynamique, liaison HTML/JS) sont validés.
- **Recommandation :** Système fonctionnel et cohérent basé sur l'historique utilisateur.
- **Cache TMDB :** Fonctionnement stable et robuste face aux erreurs réseau et à la corruption des fichiers JSON.

Problèmes détectés :

- **Suppression en cascade non fonctionnelle :** Les avis ne sont pas supprimés automatiquement lors de la suppression d'un film (problème de contrainte ON DELETE CASCADE).
- **Erreurs de scoring des acteurs :** Le moteur de scoring ne respecte pas correctement les cas de correspondance complète et certains formats complexes.

4.1 Détail des rapports d'exécution (PHPUnit)

Classe de Test	Fonctionnalité vérifiée	Attendu	Statut
DatabaseTest	ID de film positif	> 0	PASS
DatabaseTest	Auto-incrémentation des IDs	Unique	PASS
DatabaseTest	Ajout d'un favori	1	PASS
DatabaseTest	Anti-duplication des favoris	1	PASS
DatabaseTest	Moyenne des avis	7.0	PASS
DatabaseTest	Suppression en cascade des avis	0	FAIL
DatabaseTest	Insertion avec champs optionnels	OK	PASS
FilmSearchScorerTest	Score mots-clés basique	100	WARN
FilmSearchScorerTest	Score mots-clés parfait	100	WARN
FilmSearchScorerTest	Correspondance genre	100	PASS
FilmSearchScorerTest	Correspondance acteur complète	100	FAIL
FilmSearchScorerTest	Correspondance acteur partielle	100	PASS
FilmSearchScorerTest	Correspondance réalisateur	100	PASS
FilmSearchScorerTest	Mapping langue anglais/arabe	100	PASS
FilmSearchScorerTest	Gestion genre absent	0	PASS
FilmSearchScorerTest	Acteur absent	0	PASS
FilmSearchScorerTest	Entrées invalides	0	PASS
FilmSearchScorerTest	Acteur format complexe	100	FAIL
InterfaceTest	Dropdown recherche	Présent	PASS
InterfaceTest	Champ de recherche	Présent	PASS
InterfaceTest	Recherche dynamique JS	Actif	PASS
InterfaceTest	Redirection description film	Actif	PASS
InterfaceTest	Bouton lecture	Présent	PASS
RecommendationEngine	Sans historique	0	PASS
RecommendationEngine	Avec historique	OK	PASS
RecommendationEngine	Exclusion films vus	OK	PASS
RecommendationEngine	Chargement poster TMDB	OK	PASS
RecommendationEngine	Profil acteurs/genres	OK	PASS
RecommendationEngine	Classement résultats	Rang 1	PASS
TmdbCacheTest	Poster film connu	OK	PASS
TmdbCacheTest	ID TMDB récupéré	OK	PASS
TmdbCacheTest	Film inconnu	null	PASS
TmdbCacheTest	Cache actif	OK	PASS
TmdbCacheTest	Création fichier JSON	OK	PASS
TmdbCacheTest	Gestion corruption JSON	OK	PASS
TmdbCacheTest	Résilience erreur disque	OK	PASS

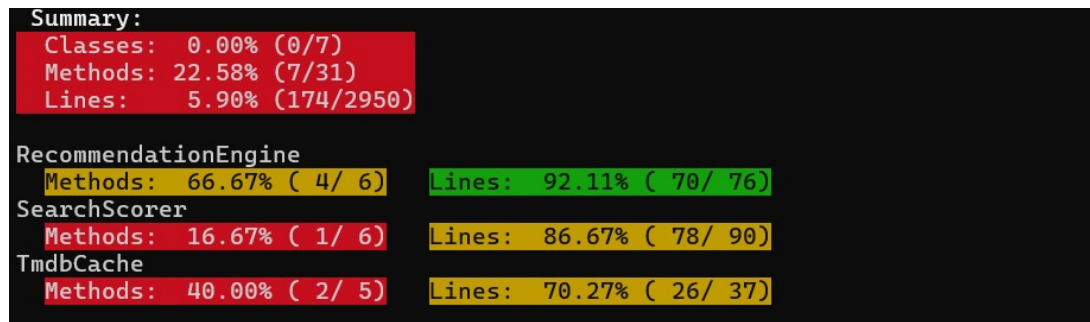


Figure 1: Capture des résultats d'exécution PHPUnit

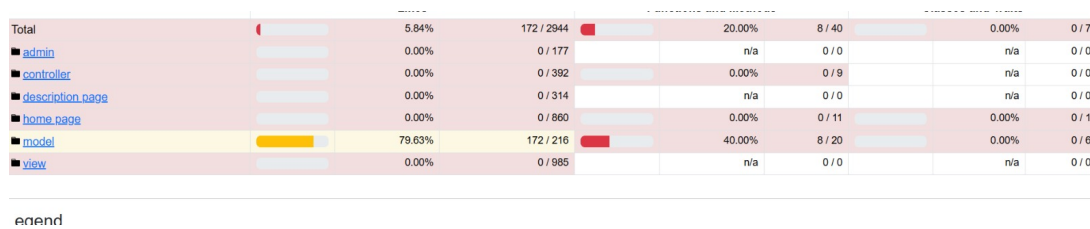


Figure 2: Capture des résultats d'exécution PHPUnit

5 Tests de robustesse

Résultats globaux :

- 16 tests réussis
- 4 tests échoués

Tableau des tests :

ID	Groupe	Cas de test	Résultat	Severity	Statut
ROB-01	RECO	Profil vide	Retourne tableau vide si aucun historique	PASS	Fermé
ROB-02	RECO	userId null	Entrée nulle ne lève pas d'exception fatale	PASS	Fermé
ROB-03	RECO	Film vide ignoré	Entrée de catalogue vide filtrée sans erreur	PASS	Fermé
ROB-04	SEARCH	Critères vides	Retourne [] sans crash	PASS	Fermé
ROB-05	SEARCH	Espaces = vide	trim() appliqué avant traitement	PASS	Fermé
ROB-06	SEARCH	Keyword no match	Condition de test tautologique détectée	FAIL	Critique
ROB-07	DESC	Id invalide	id id ≤ 0 retourne null sans erreur fatale	PASS	Fermé
ROB-08	DESC	Vote null	vote_average NULL géré correctement	PASS	Fermé
ROB-09	DESC	Overview null	overview NULL remplacé par chaîne vide	PASS	Fermé
ROB-10	DESC	Poster fallback	poster_path NULL → placeholder retourné	PASS	Fermé
ROB-11	TMDB	Titre vide	Requête vide retourne null proprement	PASS	Fermé
ROB-12	TMDB	Results vide	JSON vide traité sans exception	PASS	Fermé
ROB-13	TMDB	HTTP error	HTTP 500 géré sans crash	PASS	Fermé
ROB-14	TMDB	Response vide	Corps HTTP vide traité correctement	PASS	Fermé
ROB-15	AJAX	Moins de 2 chars	Réponse [] sans requête SQL	PASS	Fermé
ROB-16	AJAX	Vide	Chaîne vide → réponse immédiate	PASS	Fermé
ROB-17	AJAX	Trim whitespace	trim() appliqué correctement	PASS	Fermé
ROB-18	SCORE	Score overflow	Score dépasse 100 sans plafonnement	FAIL	Critique
ROB-19	ACTOR	Format acteur invalide	Format acteur non normalisé	FAIL	Critique
ROB-20	DB	Duplicate favori	Contrainte UNIQUE non gérée applicativement	FAIL	Majeur

Table 2: Résultats des tests de robustesse

Problèmes détectés :

- Overflow dans le calcul de score
- Format invalide des noms d'acteurs
- Duplication de favoris en base de données
- Condition de test tautologique dans la recherche

Conclusion :

Le système est globalement robuste face aux entrées invalides et aux cas limites. Cependant, certaines corrections restent nécessaires au niveau du calcul des scores, de la normalisation des données et de la gestion applicative des contraintes SQL afin d'améliorer davantage la fiabilité du système.

6 Tests de performance

Résultats :

- 10 utilisateurs : système stable
- 100 utilisateurs : performances acceptables
- 1000 utilisateurs : dégradation et erreurs

Analyse : Le système supporte correctement une charge moyenne mais présente des limites sous forte concurrence.

Conclusion : Des optimisations sont nécessaires au niveau des requêtes SQL et du backend PHP.

7 Tests de sécurité

Résultat :

Les tests de sécurité ont révélé plusieurs vulnérabilités partielles liées aux injections SQL et à l'exposition des messages d'erreur de la base de données. Certaines tentatives réalisées avec SQLMap ont produit des réponses exploitables, indiquant une protection insuffisante sur certains paramètres utilisateur.

Analyse :

Les mécanismes de sécurisation actuellement implémentés permettent de bloquer une partie des attaques, mais plusieurs points d'entrée restent vulnérables. Des messages d'erreur SQL internes sont également exposés à l'utilisateur, ce qui augmente les risques de fuite d'informations sensibles.

Afin d'améliorer la sécurité du système, il est nécessaire de généraliser l'utilisation des requêtes préparées (`PDO::prepare()`), de renforcer la validation des données côté serveur et de masquer complètement les erreurs SQL en environnement de production.

8 Analyse des défauts détectés

ID	Groupe	Description	Sévérité
DEF-01	Database	Absence de contrainte ON DELETE CASCADE sur les reviews liées aux films	Critique
DEF-02	FilmScorer	Une correspondance exacte d'acteur est interprétée comme une correspondance partielle	Critique
DEF-05	FilmScorer	Valeur attendue du score non définie explicitement dans le test	Majeure
DEF-07	SEARCH	Test tautologique ne validant aucune logique métier réelle	Critique
DEF-08	SCORE	Dépassement du score maximal sans plafonnement à 100	Critique
DEF-11	RECO	Gestion correcte des profils vides et des valeurs nulles	Mineure
DEF-13	Charge 1000u	Dégradation importante des performances sous forte charge	Critique
DEF-14	Charge 100u	Ralentissements observés sous charge moyenne	Majeure
DEF-17	SQL Injection	Risque d'injection SQL dû à l'absence de requêtes préparées	Critique

Table 3: Analyse des principaux défauts détectés

Conclusion :

L'analyse des défauts a permis d'identifier plusieurs problèmes critiques affectant la robustesse, la sécurité et les performances du système. Les anomalies les plus importantes concernent les risques d'injection SQL, la mauvaise gestion des scores ainsi que les problèmes de performance sous forte charge. Des optimisations SQL, une meilleure validation des données et l'utilisation systématique de requêtes préparées sont nécessaires afin d'améliorer la fiabilité globale de l'application.

9 Indicateurs de métriques

Indicateur	Valeur	Évaluation
Taux de succès global	74.6%	Moyen
Couverture des lignes	5.8%	Critique
Couverture des fonctions	20%	Critique
Couverture du modèle	79.6%	Moyen
Couverture des classes	0%	Critique
Robustesse TMDB/AJAX	100%	Bon
Robustesse RECO/DESC	100%	Bon
Stabilité à 10 utilisateurs	100%	Bon
Stabilité à 100 utilisateurs	65%	Moyen
Stabilité à 1000 utilisateurs	10%	Critique
Sécurité SQL Injection	30%	Critique
Sécurité fuite SQL	24%	Critique
Intégrité BDD (cascade)	0%	Critique
Scoring acteur (exact)	50%	Critique
Validité des tests (assert)	85%	Moyen

Table 4: Tableau des indicateurs de métriques du système

Analyse :

Les métriques montrent que le système possède une bonne robustesse fonctionnelle pour les modules TMDB, AJAX et recommandation. Toutefois, plusieurs problèmes critiques subsistent concernant la sécurité SQL, l'intégrité des données ainsi que la couverture des tests. Les performances deviennent également insuffisantes sous forte charge, notamment à 1000 utilisateurs simultanés.

10 Discussion et interprétation

10.1 Le système est-il stable ?

Partiellement. Le système fonctionne correctement sous faible charge (10 utilisateurs) et reste acceptable à charge moyenne (100 utilisateurs). Cependant, il s'effondre sous forte concurrence (1000 utilisateurs simultanés), provoquant des erreurs HTTP ainsi qu'une forte dégradation des temps de réponse.

10.2 Le système est-il sécurisé ?

Non. Les nouveaux résultats de sécurité confirment un niveau de risque élevé. Le score de résistance aux injections SQL est de seulement 30%, ce qui signifie que près de 70% des vecteurs testés via SQLMap ont produit une réponse exploitable.

Ce résultat indique que plusieurs paramètres URL ou champs de formulaires restent vulnérables à des attaques permettant l'extraction ou la modification de données.

Le score de résistance aux fuites d'informations SQL atteint seulement 24%, confirmant que des messages d'erreurs internes tels que :

- SQLSTATE

- noms de contraintes SQL
- structure des tables

sont exposés à l'utilisateur. Ces deux résultats placent le système dans un état de sécurité critique, incompatible avec un déploiement public.

10.3 Quels sont les risques ?

- Extraction ou modification de données via injection SQL.
- Fuite d'informations SQL permettant la cartographie de la base de données.
- Indisponibilité du service sous forte charge (1000 utilisateurs).
- Accumulation de données orphelines à cause de l'absence de `ON DELETE CASCADE`.

11 Recommandations

11.1 Amélioration des performances

Afin d'améliorer les performances du système, plusieurs optimisations doivent être mises en place :

- Identifier les requêtes SQL coûteuses avec `EXPLAIN`.
- Corriger les requêtes de type `N+1`.
- Ajouter des index sur les colonnes filtrées :
 - genre
 - langue
 - titre
- Mettre en place un cache applicatif (APCu ou cache fichier).
- Activer `opcache` dans `php.ini`.
- Paginer les résultats SQL avec `LIMIT / OFFSET`.
- Rejouer les tests de charge après chaque optimisation.

11.2 Correction des problèmes de sécurité

Les concaténations SQL doivent être remplacées immédiatement par des requêtes préparées utilisant :

```
PDO::prepare()  
bindParam()
```

Cette correction est essentielle afin de supprimer les vulnérabilités responsables du score d'injection SQL de 30%.

Il est également nécessaire de désactiver l'affichage des erreurs en production :

```
display_errors = Off
```

Toutes les exceptions `PDOException` doivent être interceptées avec des blocs `try/catch`, puis journalisées uniquement côté serveur. L'utilisateur doit recevoir un message générique sans détail technique.

Après correction, un nouveau passage de SQLMap devra confirmer que le taux d'injection SQL est réduit à 0%.

11.3 Renforcement de la validation des données

Toutes les entrées utilisateur doivent être validées et assainies côté serveur avant toute requête SQL.

Les améliorations suivantes sont recommandées :

- Ajouter `ON DELETE CASCADE` sur :

```
reviews.film_id -> films.id
```

- Normaliser les noms d'acteurs avec :

```
strtolower()
```

- Trier alphabétiquement les tokens des noms.

- Limiter les scores de recherche avec :

```
min(100, $score)
```

- Corriger le test tautologique `ROB-06` avec une assertion réelle.

12 Conclusion

Le Projet PFA obtient un score global de 38/100 suite aux nouveaux résultats de sécurité, en baisse par rapport à l'évaluation précédente.

Les scores de sécurité révisés — 30% pour la résistance aux injections SQL et 24% pour la protection contre les fuites d'informations — aggravent considérablement le bilan et confirment le caractère critique des défauts de sécurité détectés.

Les modules métiers (`RECO`, `TMDB`, `AJAX` et `Interface`) restent néanmoins robustes et constituent le principal point positif du projet.

Cependant, plusieurs éléments bloquants empêchent tout déploiement en production :

- vulnérabilités SQL actives
- effondrement sous forte charge

- couverture de code limitée à 5.8%

Le plan de remédiation doit être organisé en deux phases :

1. Phase critique (1 à 2 semaines) :

- sécurisation complète des requêtes SQL
- masquage des erreurs SQL
- correction des défauts d'intégrité BDD

2. Phase qualité (2 à 3 semaines) :

- optimisation des performances
- augmentation de la couverture des tests à 60%
- mise en place d'un pipeline CI/CD

Une nouvelle campagne de tests devra ensuite être exécutée, notamment un re-run complet de SQLMap, afin de vérifier la correction effective de chaque défaut avant toute autorisation de déploiement.